Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

Implementar 3 funções para gerenciar a sincronização de uma thread de metrô com várias threads de passageiro. Implementei essas 3 funções e ainda uma para inicializar o mutex e variáveis de condição da struct do metrô.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

station_load_car	OK	
station_wait_for_car	OK	
station_on_board	OK	

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make sync_car
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.c → Implementação das 3 funções e da inicialização a serem usadas no runner**
2. **sync_car.h → Informações sobre o struct para ambos os arquivos .c e funções para o runner**

Decisões técnicas:

1. **Fazer um arquivo sync_car.h e sync_car.c → O runner pede sync_car.h → A implementação é num arquivo diferente do runner**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

S1	Fazer tudo com IA	Depressão	Eu não entendi o código
S2	Fazer o código normal	Nada de muito especial na implementação. Todas as operações ocorrem dentro de locks de mutexes exceto a checagem de se count <= 0 em station_load_car	
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

1	S2	Rodar	`Error: Failed to complete after 1 seconds. Something's wrong, or your system`	Erro na condição do wait do metrô, com ele esperando se estivesse SEM passageiros no processo de entrar, em vez de COM. Espera infinita.	Domingo	Log 27

			`is terribly slow. Possible error hint: [station_load_car() did not return immediately when no waiting passengers]`			
2	S2	Rodar depois de ajeitar o anterior	`Error: station_load_car failed`	Quando o metrô dava broadcast, todas as pessoas entravam de uma vez porque o metrô só atualizava sua variável de cheio após o signal de uma pessoa ter sentado nele. Isso fazia todas as pessoas entrarem na contagem de “entrando” do metrô, que o impedia de sair por não ser 0, e também sem as pessoas poderem sentar pelo metrô ter atingido a capacidade máxima.	Domingo	Log 38-422
3	S2	Rodar depois de ajeitar o anterior	`Looks good!`		Domingo	Log 430+

4						
5						
6						
7						
8						

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

• Erro principal: o momento do bug mais relevante da semana.

• Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

```
Error: Failed to complete after 1 seconds. Something's wrong, or your system is terribly slow. Possible error hint: [station_load_car() did not return immediately when no waiting passengers]
```

```c
while (!(station->passengersEntering) && (station->passengersOutside || !(station->maxCapacity))) {
    pthread_cond_wait(&(station->onboard), &(station->mutex));
    if (station->passengersInside == count) {
        station->maxCapacity = 1;
    }
}
```

No metrô - condição do while errada e atualização da capacidade também só ocorrendo depois do signal. Também não conta as pessoas em estado de entrando na capacidade máxima.

```
car entering station with 6 free seats
Esperei - I: 0 - O: 100 - E: 0
Acordei - I: 1 - O: 0 - E: 99
Esperei - I: 1 - O: 0 - E: 99
Acordei - I: 2 - O: 0 - E: 98
Esperei - I: 2 - O: 0 - E: 98
Acordei - I: 3 - O: 0 - E: 97
Esperei - I: 3 - O: 0 - E: 97
Acordei - I: 4 - O: 0 - E: 96
Esperei - I: 4 - O: 0 - E: 96
Acordei - I: 5 - O: 0 - E: 95
Esperei - I: 5 - O: 0 - E: 95
Acordei - I: 6 - O: 0 - E: 94
Esperei - I: 6 - O: 0 - E: 94
Error: station_load_car failed
```

I = Inside, O = Outside, E = Entering

```
while (!(station->arrived) || station->maxCapacity) {
    pthread_cond_wait(&(station->arrival), &(station->mutex));
}
```

No passageiro - checa um valor de capacidade máxima que só iria atualizar depois de sentar, em vez de atualizar a cada passageiro que começa a entrar

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Tive alguma ideia da quantidade de funções e escopo do código, da utilização de variáveis de pthread dentro de structs, da existência da função station_init e, principalmente, utilizei uma variável do número de pessoas atualmente entrando no metrô que tinha sido adicionada nesse código.

Prompts principais (liste, sem história):

“Você pode fazer a tarefa solicitada?”, com o documento da implementação 5 e o código do runner mandados anteriormente.

O que validei manualmente e como:

Não usei esse código, exceto pela lógica de atualização da variável station->passengersEntering. Rodei o meu código e, mesmo essa sendo a variável que causou mais problemas, a parte da atualização em si dela não causou nenhum erro.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Não melhoraria nada. Juntei múltiplos arquivos .c utilizando um arquivo .h pela primeira vez, e organizei o makefile para essa estrutura pela primeira vez também.

9. Checklist Final de Entrega

Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	Não tenho controle disso
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico. Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Não vejo nada no código que iria melhorar. Poderia aprender a como usar flags ou comandos genéricos no Makefile.

Regras de Integridade (não mudam)

• Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
• Contradição entre prints, log e relato = relatório perde credibilidade.
• Indício de evidência fabricada = entrega invalidada.